

Discrete Logarithm

离散对数

定义

前置知识：[阶与原根](#)。

离散对数的定义方式和对数类似。取有原根的正整数模数 m ，设其一个原根为 g 。对满足 $0 \leq x < m$ 的整数 x ，我们知道必存在唯一的整数 y 使得

我们称这个 y 为以 g 为底，模 m 的离散对数，记作 $\log_g x$ ，在不引起混淆的情况下可记作 x 。

显然， $\log_g g = 1$ 。

性质

离散对数的性质也和对数有诸多类似之处。

▼ 性质

设 g 是模 m 的原根， x ， y 则：

1.

进而

2. 若 h 也是模 m 的原根，则

3.

▼ 证明

1.

2. 令 $x = g^y$ ，则 $\log_g x = y$ 。又令 $h = g^a$ ，则 $\log_h x = \log_{g^a} x$ 。

故，即

3. 注意到

大步小步算法

目前离散对数问题仍不存在多项式时间经典算法（离散对数问题的输入规模是输入数据的位数）。在密码学中，基于这一点人们设计了许多非对称加密算法，如 [Ed25519](#)。

在算法竞赛中，BSGS（baby-step giant-step，大步小步算法）常用于求解离散对数问题。形式化地说，对 x ，该算法可以在 $O(\sqrt{m})$ 的时间内求解

其中 x 。方程的解 x 满足 $x \equiv a \pmod{m}$ 。（注意 m 不一定是素数）

算法描述

令 $x = a + km$ ，其中 k ，则有 $a + km \equiv a \pmod{m}$ ，稍加变换，则有 $km \equiv 0 \pmod{m}$ 。

我们已知的是 a ，所以我们可以先算出等式右边的 a 的所有取值，枚举 a ，用 `hash/map` 存下来，然后逐一计算 a ，枚举 a ，寻找是否有与之相等的 a ，从而我们可以得到所有的 a 。

注意到 a 均小于 m ，所以时间复杂度为 $O(m)$ ，用 `map` 则多一个 $\log$ 。

► 为什么要求 a 与 m 互质

进阶篇

对 a ， m ，求解

该问题可以转化为 BSGS 求解的问题。

由于式子中的模数 m 是一个质数，那么 m 一定存在一个原根 g 。因此对于模 m 意义下的任意的数 a 有且仅有一个数 x 满足 $a \equiv g^x \pmod{m}$ 。

方法一

我们令 $x = yz$ ， z 是 m 的原根（我们一定可以找到这个 z 和 m ），问题转化为求解 $g^{yz} \equiv a \pmod{m}$ 。稍加变换，得到

于是就转换成了 BSGS 的基本模型了，可以在 $O(\sqrt{m})$ 解出 y ，这样可以得到原方程的一个特解。

方法二

我们仍令 $x = yz$ ，并且设 $z = m - 1$ ，于是我们得到

方程两边同时取离散对数得到

我们可以通过 BSGS 求解 y 得到 y ，于是这就转化成了一个线性同余方程的问题。这样也可以解出 y ，求出 x 的一个特解。

找到所有解

在知道 x 的情况下，我们想得到原问题的所有解。首先我们知道 $x \equiv a \pmod{m}$ ，于是可以得到

于是得到所有解为

对于上面这个式子，显然有 $x \equiv a \pmod{m}$ 。因此我们设 $x = a + km$ ，得到

这就是原问题的所有解。

实现

下面的代码实现的找原根、离散对数解和原问题所有解的过程。

► 参考代码

扩展篇（扩展 BSGS）

对，求解

其中 不一定互质。

当 时，在模 意义下 存在逆元，因此可以使用 BSGS 算法求解。于是我们想办法让他们变得互质。

具体地，设 . 如果，则原方程无解。否则我们把方程同时除以，得到

如果 和 仍不互质就再除，设 . 如果，则方程无解；否则同时除以 得到

同理，这样不停的判断下去，直到 .

记，于是方程就变成了这样：

由于，于是推出 . 这样 就有逆元了，于是把它丢到方程右边，这就是一个普通的 BSGS 问题了，于是求解 后再加上就是原方程的解啦。

注意，不排除解小于等于 的情况，所以在消因子之前做一下 枚举，直接验证，这样就能避免这种情况。

习题

- [SPOJ MOD](#) 模板
- [SDOI2013 随机数生成器](#)
- [SGU261 Discrete Roots](#) 模板
- [SDOI2011 计算器](#) 模板
- [Luogu4195 【模板】exBSGS/Spoj3105 Mod](#) 模板
- [Codeforces - Lunar New Year and a Recursive Sequence](#)
- [LOJ6542 离散对数](#) index calculus 方法，非模板

本页面部分内容以及代码译自博文 [Дискретное извлечение корня](#) 与其英文翻译版 [Discrete Root](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

参考资料

1. [Discrete logarithm - Wikipedia](#)
 2. 潘承洞，潘承彪。初等数论。
 3. 冯克勤。初等数论及其应用。
-

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[H-J-Granger](#), [StudyingFather](#), [Alpha1022](#), [Chrogeek](#), [ChungZH](#), [countercurrent-time](#), [Entertainer](#), [FFjet](#), [Great-designer](#), [Henry-ZHR](#), [hly1204](#), [hsfzLZH1](#), [iamtwz](#), [lr1d](#), [isdanni](#), [Kelatte](#), [ksyx](#), [Lampese](#), [Marcythm](#), [MegaOwler](#), [ouuan](#), [purple-vine](#), [sshwy](#), [Steauk](#), [stevebraveman](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#), [xyf007](#), [YOYO-UIAT](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用